



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Documentazione di progetto di **Software Architecture
Design**

Gruppo D7 Task R8

Anno Accademico 2025/2026

Gruppo:

Fabio Davide Auriemma

Marco Canonico

Luca Carraturo

Angela Dalia

Indice

Gruppo D7	1
0.1 Gruppo di lavoro	1
0.2 Task R8	1
0.3 Approccio di sviluppo seguito	2
1 Analisi dei Requisiti	4
1.1 Modello del Dominio	4
1.2 Requisiti non Funzionali	6
1.3 Analisi dell’Impatto	7
1.3.1 Modifiche all’API Gateway	7
1.3.2 Modifiche all’Infrastruttura di Osservabilità . .	8
1.3.3 Modifiche ai Servizi	9
2 Progettazione Soluzione	10
2.1 Decisioni Progettuali	10
2.1.1 Standardizzazione e Gestione Centralizzata degli Errori	11
2.1.2 Ottimizzazione dell’Osservabilità (Agent vs Li- brary)	13
2.1.3 Ottimizzazione del tempo di risposta per T8 . .	14

2.2	Dinamica dell'Architettura	15
2.2.1	Diagramma di Sequenza: Circuit Breaker . . .	15
2.2.2	Diagramma a Stati: Circuit Breaker	16
2.2.3	Flusso di Esecuzione del Rate Limiting	18
2.2.4	Analisi Dinamica: Pipeline di Osservabilità . .	21
2.3	Issue e Anti-pattern	21
3	Implementazione e Struttura del Progetto	24
3.1	Moduli Aggiunti o Modificati	24
3.2	Implementazione del Circuit Breaker	27
3.2.1	Gestione delle Dipendenze	28
3.2.2	Configurazione del Routing	28
3.2.3	Configurazione dei Parametri di Resilienza . . .	30
3.3	Ottimizzazione e Configurazione di EvoSuite	33
3.3.1	Configurazione della JVM e Parametri di EvoSuite	33
3.3.2	Processo di Build Maven	35
3.3.3	Adattamento del Data Transfer Object	35
3.4	Implementazione del Rate Limiter	35
3.4.1	Configurazione del Key Resolver	36
3.4.2	Configurazione del Routing e Parametri	37
3.5	Configurazione Prometheus e Otel	39
3.5.1	Dipendenze necessarie	39
3.5.2	File .properties	41
3.5.3	File resources/logback-spring.xml	42
3.5.4	File docker-compose.yml	43
3.5.5	Configurazione prometheus_config.yml	44

3.6	Configurazione e Verifica di Grafana	45
3.6.1	Verifica dei Data Sources	45
3.6.2	Importazione delle Dashboard	46
4	Deployment	48
4.1	Deployment Diagram	48
5	Testing	49
5.1	Test Effettuati	49
5.1.1	Validazione del Rate Limiter	50
5.1.2	Validazione del CircuitBreaker	54
5.1.3	Test di Integrazione	56
6	Conclusioni	60
6.1	Sviluppi Futuri e Raccomandazioni	60
	Bibliografia	62

Gruppo D7

0.1 Gruppo di lavoro

Componente	Matricola	Mail
Fabio Davide Auriemma	M63001848	david.auriemma@studenti.unina.it
Marco Canonico	DE9000168	marco.canonico@studenti.unina.it
Luca Carraturo	DE9000145	lu.carraturo@studenti.unina.it
Angela Dalia	DE9000055	ange.dalia@studenti.unina.it

Versione di partenza: Branch D7_R8

Versione consegnata: Repo consegna

0.2 Task R8

L'API Gateway prevede un filtro di logging delle richieste (e relative risposte), nonché dei meccanismi di Rate Limit e Circuit Breaker attivabili e configurabili tramite il file `application.yml`.

- Studiare e comprendere l'attuale implementazione di Rate Limit

e Circuit Breaker dell'API Gateway, partendo dalla documentazione disponibile nella WIKI;

- Verificare l'utilità di questi meccanismi e definire più correttamente i parametri di configurazione di Rate Limit e Circuit Breaker rispetto alle rotte verso T7 e T8, che rappresentano i due colli di bottiglia dell'applicazione;
- Sarebbe auspicabile definire il formato di risposta restituito dai meccanismi di Rate Limit e Circuit Breaker per permettere una gestione uniforme dei messaggi restituiti in ogni modulo.

0.3 Approccio di sviluppo seguito

Il gruppo ha seguito un processo di sviluppo iterativo. In seguito all'assegnazione del task si è incontrato una prima volta per definire sia gli obiettivi finali che quelli del primo sprint. È stato utilizzato Notion sia in una prima fase di brainstorming, effettuata anche mediante tecnica del post-it, che in fase di preparazione e gestione del backlog degli sprint.

Backlog

sprint

Name	Ev. Output	Prio	Assigned To	status
Comprendere ed eventualmente migliorare il caching Redis		Bassa		Not started
Configurazione Open Telemetry Grafici su Grafana	Diagramma di seq usando il tracing	Alta	Luca Carraturo Marco	Done
Implementare il CB su T7 e T8		Alta	Angela Dalia	Done
Rate Limting e simulazione token di autenticazione	Report delle richieste effettuate, con tempo di risposta	Alta	Fabio Auriemma	In progress
Documentazione	Documento	Alta	Marco Luca Carraturo Angela Dalia Fabio Auriemma	In progress
Implementare RabbitMQ in T7 e T8		Media	Luca Carraturo	Not started
Scalabilità orizzontale		Bassa		Not started
Gestione caso errore		Alta	Marco	Done
Trovare falle sicurezza		Bassa	Marco	Not started
Migliorare i tempi di attesa configurando diversamento Evosuite		Bassa	Angela Dalia	Done

Figure 1: Backlog implementato in Notion

Chapter 1

Analisi dei Requisiti

In seguito all'analisi della richiesta sono stati sintetizzati gli scenari di qualità rappresentati nelle Figure: 1.2, 1.3, 1.4, 1.5

1.1 Modello del Dominio

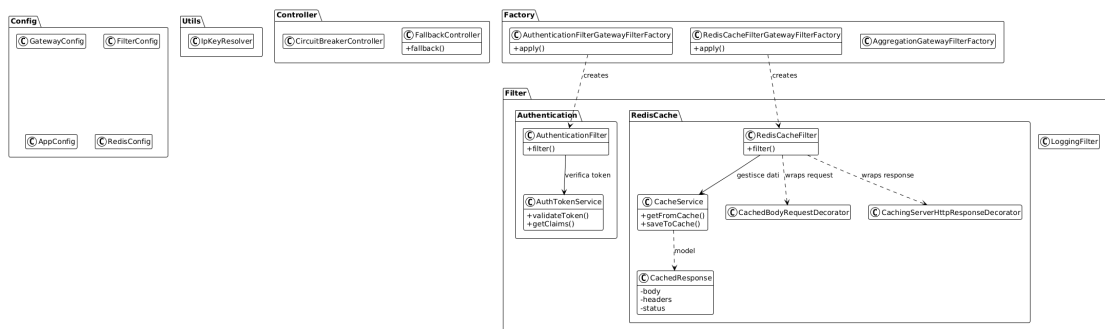


Figure 1.1: Class Diagram dell'apiGateway

Il diagramma rappresenta l'architettura del modulo **API Gateway** basato su **Spring Cloud Gateway**, dove la logica di instradamento è dichiarativa e definita nel file `application.yml`. Le classi sono or-

ganizzate in package che separano la configurazione dalla logica operativa:

Package Config Rappresenta il punto di ingresso per la configurazione del contesto Spring. Qui risiedono le classi annotate con `@Configuration` (come `AppConfig` e `FilterConfig`) responsabili della definizione e dell'iniezione dei Bean infrastrutturali necessari al funzionamento del gateway e alla connessione con servizi esterni come Redis.

Package Factory Costituisce il ponte tra la configurazione dichiarativa (YAML) e il codice Java. Utilizzando il pattern *Factory*, queste classi (es. `AuthenticationFilterGatewayFilterFactory`) permettono a Spring di istanziare dinamicamente i filtri corretti quando vengono richiesti nelle definizioni delle rotte.

Package Filter Contiene il core della logica di business del Gateway. È suddiviso in sotto-moduli specifici per funzionalità, come **Authentication** (per la sicurezza e validazione token) e **RedisCache** (per l'ottimizzazione delle performance), oltre ai filtri globali per il logging. Qui avviene la manipolazione effettiva delle richieste e delle risposte.

Package Controller & Utils Forniscono il supporto all'infrastruttura REST. I Controller gestiscono gli endpoint di servizio (come i fallback in caso di errore), mentre il package Utils contiene com-

ponenti ausiliari, come i **KeyResolver**, necessari per identificare i client nelle logiche di limitazione del traffico (Rate Limiting).

1.2 Requisiti non Funzionali

ID	Descrizione del Requisito
RNF-01	Definire più correttamente i parametri di configurazione del Circuit Breaker rispetto alle rotte verso T7 e T8, che rappresentano i due colli di bottiglia dell'applicazione.
RNF-02	Definire più correttamente i parametri di configurazione del Rate Limit rispetto alle rotte verso T7 e T8, verificando l'utilità di questi meccanismi.
RNF-03	Definire il formato di risposta restituito dai meccanismi di Rate Limit e Circuit Breaker per permettere una gestione uniforme dei messaggi restituiti in ogni modulo.
RNF-aggiunto	Il sistema deve supportare la manutenibilità e il debugging attraverso uno stack di osservabilità centralizzato permettendo l'identificazione delle cause di errore senza necessità di accesso diretto ai container.

Table 1.1: Tabella dei Requisiti Non Funzionali

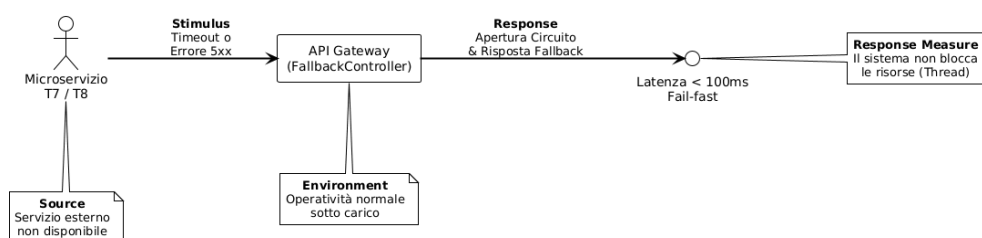


Figure 1.2: Scenario di Qualità per la Resilienza (Circuit Breaker)

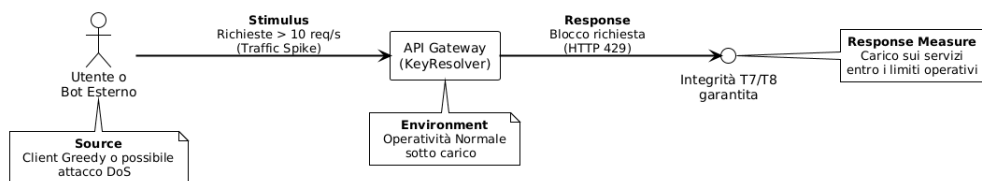


Figure 1.3: Scenario di Qualità per la Performance (Rate Limiting)

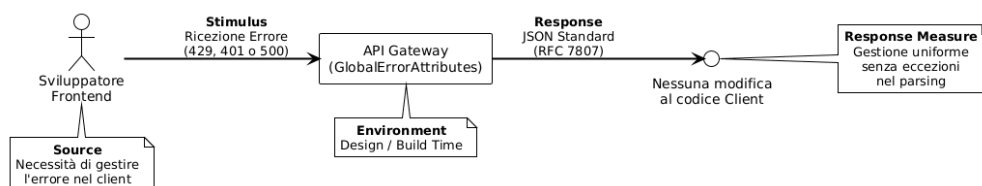


Figure 1.4: Scenario di Qualità per la Manutenibilità (Uniform Response)

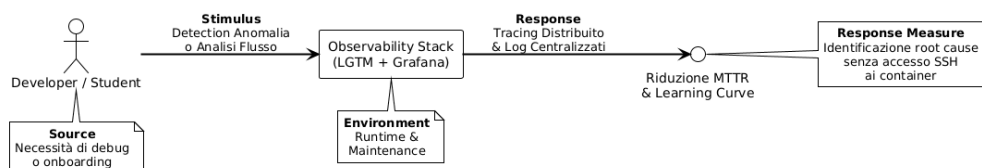


Figure 1.5: Scenario di Qualità per la Manutenibilità (Osservabilità LGTM)

1.3 Analisi dell'Impatto

1.3.1 Modifiche all'API Gateway

Oltre alle logiche di resilienza, sono state modificate le configurazioni di routing e i filtri di sicurezza personalizzati.

File Modificati/Creati	Obiettivo
IpKeyResolver.java PrincipalNameKeyResolver.java	Identificazione client per Rate Limiting.
GatewayConfig.java	Configurazione Principal-NameKeyResolver.
FallbackController.java GlobalErrorAttributesFilter.java	Gestione Fallback e uniformazione errori.
AuthenticationFilter.java	Logica custom filtro autenticazione.
pom.xml application.yml	Dipendenze Resilience4j, timeout, Rate Limiter e Circuit Breaker.

Table 1.2: Dettaglio modifiche API Gateway

1.3.2 Modifiche all’Infrastruttura di Osservabilità

Include i file per il Collector, i backend di storage e le dashboard, oltre alla configurazione generale dei container.

File Modificati/Creati	Obiettivo
otel_collector_config.yml	Configurazione pipeline OTel (Receiver, Processor, Exporter).
prometheus_config.yml tempo_config.yml datasource.yml	Configurazione backend metriche/tracce e sorgenti dati Grafana.
Overview.json ServiceDetail.json LogsInvestigation.json	Definizione JSON delle Dashboard Grafana.
docker-compose.yml	Orchestrazione container dello stack di monitoraggio.

Table 1.3: Dettaglio modifiche dell’observability stack

1.3.3 Modifiche ai Servizi

Questa sezione copre gli interventi sui singoli servizi (T1, T4, T5, T8, T23), inclusi script di database e di automazione deploy.

File Modificati/Creati	Obiettivo
pom.xml (T1, T4, T5, T8) application.properties (T4)	Aggiunta dipendenze e configurazione per tracciamento distribuito.
AuthTokenFilter.java WebSecurityConfig.java	Uniformazione sicurezza per garantire lo scraping di Prometheus (T1, T5, T23).
CoverageService.java (T8) BuildResponse.java (T8)	Intervento sulla logica di business del servizio T8.
init.sql (T23)	Script di inizializzazione database per il servizio T23.

Table 1.4: Dettaglio modifiche Microservizi e Script

Chapter 2

Progettazione Soluzione

2.1 Decisioni Progettuali

Per quanto riguarda i pattern di resilienza (*Circuit Breaker* e *Rate Limiter*), le scelte progettuali sono state dettate quasi interamente dallo stack tecnologico. Utilizzando Spring Cloud Gateway, l'integrazione con la libreria **Resilience4j** rappresenta la soluzione standard per gestire la stabilità del sistema, rendendo superflua l'implementazione di soluzioni custom.

Il lavoro di progettazione si è quindi concentrato su tre aspetti dove era necessario ottimizzare il comportamento dell'architettura: la gestione centralizzata degli errori, l'efficienza del sistema di monitoraggio e il miglioramento delle performance di T8.

2.1.1 Standardizzazione e Gestione Centralizzata degli Errori

Ogni servizio tende a restituire messaggi di errore con formati propri, status code inconsistenti o body di risposta non strutturati (es. semplice testo). L'eterogeneità nelle risposte è per i client del **apiGateway** un problema che genera:

1. **Interoperabilità ridotta:** Il client è costretto a implementare logiche di parsing specifiche per ogni microservizio per comprendere la natura dell'errore.
2. **Difficoltà di automazione:** L'assenza di un formato *machine-readable* standard impedisce al client di reagire programmaticamente agli errori (es. distinguere un errore di validazione da un errore di sistema).

Per risolvere queste criticità, la progettazione ha previsto l'adozione dello standard IETF **RFC 9457** [Nottingham and Wilde, 2023] (che ha sostituito il precedente RFC 7807), noto come "*Problem Details for HTTP APIs*". Questo standard definisce un formato JSON univoco per trasportare i dettagli di un errore in una risposta HTTP, disaccoppiando l'errore dalla sua rappresentazione specifica. Lo standard impone una struttura comune che permette ai client di identificare univocamente il tipo di problema.

Listing 2.1: Esempio di struttura standard RFC 9457

```
{
  "type": "about:blank",
  "title": "Too Many Requests",
  "status": 429,
  "detail": "Il limite di richieste è stato superato.
    Riprova più tardi.",
  "instance": "/compile/jacoco/coverage/player"
}
```

I campi obbligatori sono:

- **type**: Un URI che identifica il tipo di problema (utile per la documentazione automatica).
- **title**: Un breve sommario leggibile del problema.
- **status**: Lo status code HTTP generato dal server di origine.

Per implementare questo standard la soluzione progettuale prevede l'introduzione di un componente centralizzato a livello del Gateway: il **Global Error Attributes Filter**. Il funzionamento logico del filtro è descritto nel Diagramma di Attività in Figura 2.1. Il filtro agisce come un wrapper sulla risposta HTTP: monitora lo stato in uscita e, qualora rilevi errori come problemi di autenticazione o limiti di traffico superati, normalizza l'output sostituendo il corpo della risposta con un oggetto JSON strutturato secondo le specifiche RFC 9457.

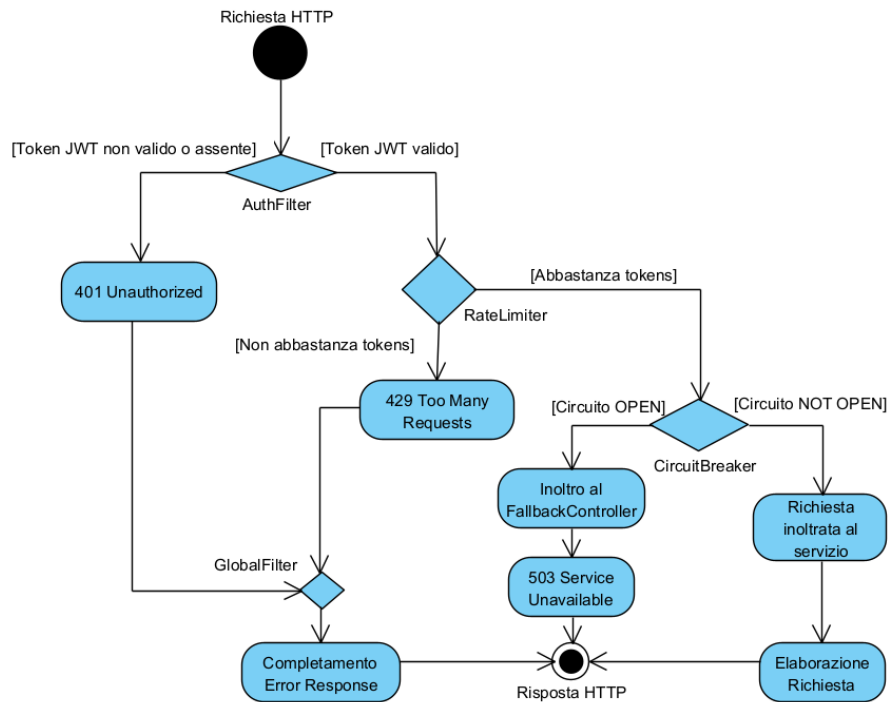


Figure 2.1: Activity Diagram del Global Error Filter

2.1.2 Ottimizzazione dell'Osservabilità (Agent vs Library)

Una decisione critica ha riguardato la strategia per il tracciamento distribuito. Inizialmente è stato valutato l'utilizzo dell'**OpenTelemetry Java Agent**, che permette l'auto-strumentazione dei servizi senza modificare il codice. Tuttavia, l'Agent introduce un *overhead* di memoria RAM significativo su ogni container a runtime. Per ottimizzare l'uso delle risorse, abbiamo optato per la **strumentazione tramite libreria** (usando Micrometer Tracing integrato nelle dipendenze). Con questo approccio, i microservizi rimangono leggeri e delegano il carico di elaborazione ed esportazione dei dati a un unico componente infras-

trutturale, l'**OpenTelemetry Collector**.

2.1.3 Ottimizzazione del tempo di risposta per T8

Il microservizio T8, responsabile del calcolo della copertura con EvoSuite, è stato oggetto di un intervento di refactoring mirato a migliorare la scalabilità e i tempi di risposta. L'analisi ha evidenziato l'opportunità di ottimizzare il ciclo di vita della build Maven, precedentemente suddiviso in più fasi, e di adeguare la configurazione Java alle risorse hardware del server (32GB di RAM), che la configurazione standard non sfruttava appieno.

Per massimizzare l'efficienza, abbiamo adottato la strategia "High-Memory Sequential". Abbiamo razionalizzato la pipeline di build unificando i comandi in un singolo step atomico, riducendo i tempi di avvio. Sul fronte infrastrutturale, abbiamo riconfigurato la JVM allocando 6GB di RAM al processo EvoSuite e adottando il ParallelGC, garantendo così un'esecuzione fluida e priva di colli di bottiglia di memoria. Inoltre, abbiamo snellito la logica rimuovendo il calcolo di metriche non visualizzate. Il risultato è un servizio più performante e stabile, capace di valorizzare al meglio l'hardware a disposizione.

2.2 Dinamica dell'Architettura

In questa sezione viene analizzato il comportamento dinamico del sistema a seguito di tre interventi chiave sull'API Gateway: l'introduzione del pattern Circuit Breaker, la modifica della configurazione del Rate Limiter e l'adozione di un Global Filter per la gestione degli errori. Vengono descritti i flussi di interazione tra i componenti in caso di guasto e la logica interna di transizione degli stati del circuit breaker.

2.2.1 Diagramma di Sequenza: Circuit Breaker

La figura 2.2 illustra il flusso di una richiesta HTTP inviata da un client (es. Postman o Frontend) verso il microservizio T7, protetto dal Circuit Breaker.

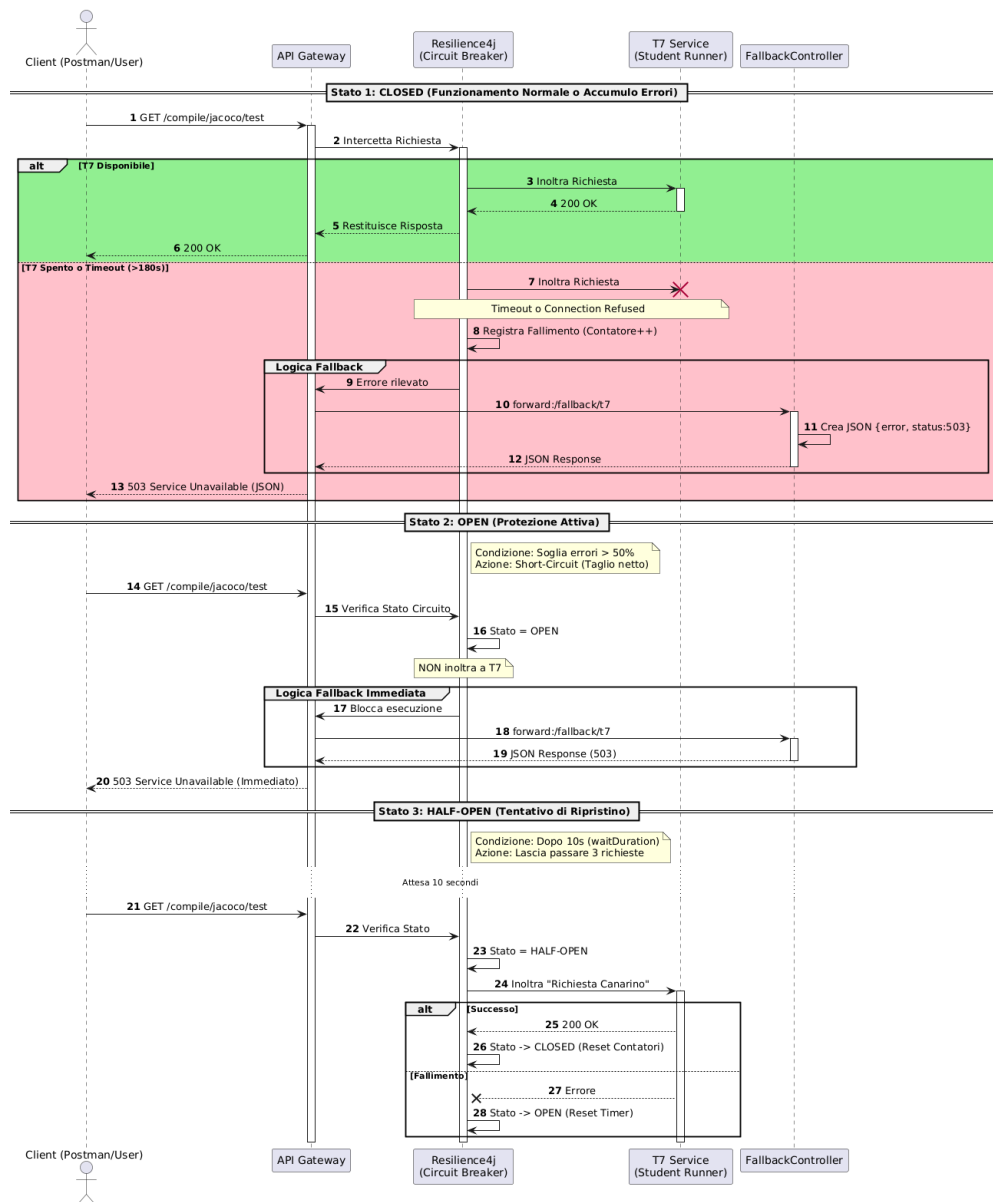


Figure 2.2: Richiesta con Circuit Breaker e Fallback su T7

2.2.2 Diagramma a Stati: Circuit Breaker

Il comportamento del Circuit Breaker è governato da una macchina a stati finiti, configurata nel file `application.yml`. Il diagramma in figura 2.3 descrive le transizioni di stato basate sulle metriche di traffico verso i servizi T7 e T8.

Gli stati e le transizioni sono definiti come segue:

- **CLOSED:** Stato operativo normale. Il traffico fluisce verso il microservizio. Se il tasso di fallimento supera la soglia del 50% (su una finestra di 10 richieste), avviene la transizione allo stato OPEN.
- **OPEN:** Stato di protezione. Le richieste vengono bloccate per un periodo di *Wait Duration*, configurato a 10 secondi, al termine del quale avviene la transizione automatica allo stato HALF-OPEN.
- **HALF-OPEN:** Stato di verifica. Il sistema consente il passaggio di un numero limitato di richieste di prova (3 richieste).
 - Se le richieste hanno successo, il servizio è considerato ripristinato e si torna allo stato **CLOSED**.
 - Se le richieste falliscono, il servizio è ancora instabile e si ritorna allo stato **OPEN**, riavviando il timer di attesa.

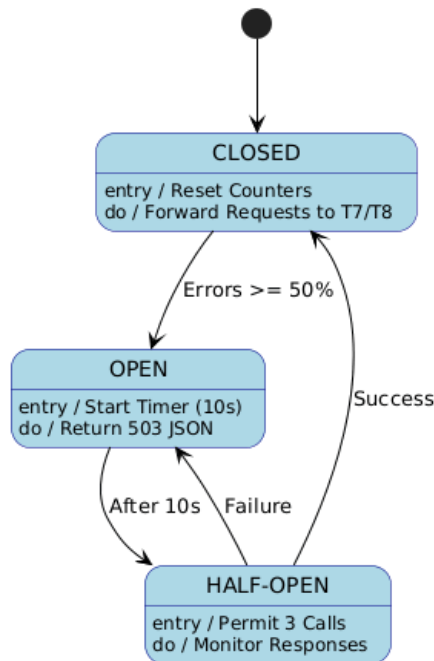


Figure 2.3: Macchina a stati del Circuit Breaker

2.2.3 Flusso di Esecuzione del Rate Limiting

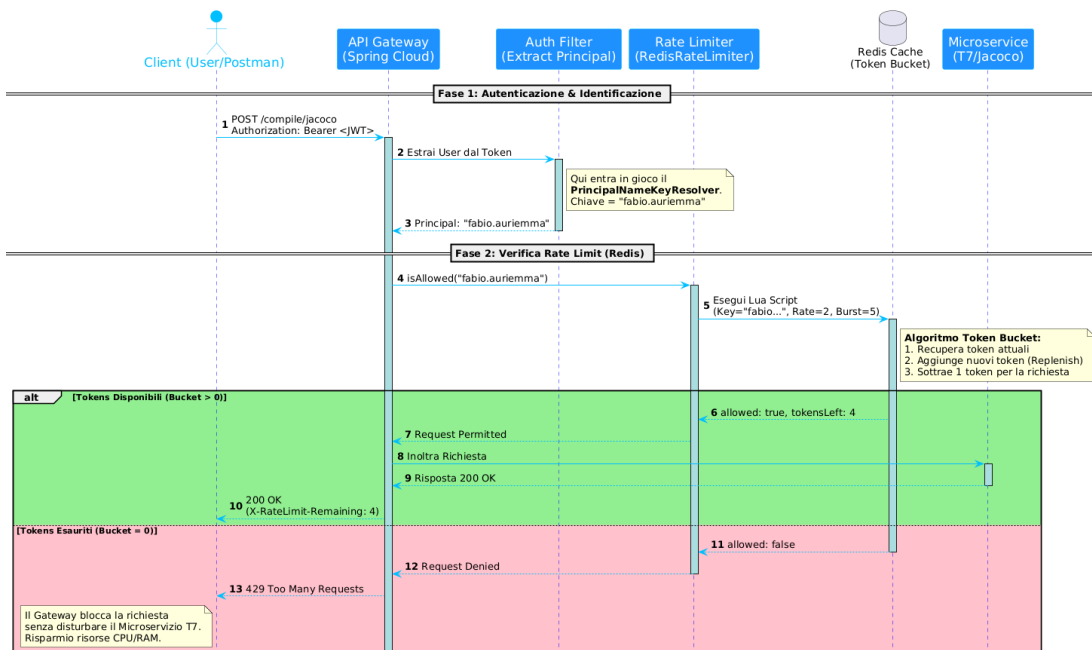


Figure 2.4: Diagramma di sequenza del Rate Limiting

Il processo, come mostrato in figura 2.4, si articola nelle seguenti fasi principali:

1. Autenticazione e Identificazione del Principal: Il flusso inizia quando il client, un utente o Postman, invia una richiesta protetta (es. `POST /compile/jacoco`) all'API Gateway. La richiesta include un **JSON Web Token (JWT)** nell'header `Authorization`. L'API Gateway intercetta la richiesta e, prima di applicare qualsiasi logica di limitazione, deve identificare il richiedente. Il componente di autenticazione estrae il *Principal* (il nome utente univoco) dal payload del JWT. Nel diagramma, l'utente viene identificato come `"fabio.auriemma"`. Questa stringa diventa la chiave univoca utilizzata per il conteggio delle richieste in Redis, garantendo che i limiti siano applicati all'identità dell'utente indipendentemente dall'indirizzo IP di provenienza.

2. Verifica della quota su Redis Una volta identificato l'utente, il Gateway interpella il servizio di Rate Limiter. Quest'ultimo esegue una chiamata atomica verso il database Redis. Redis implementa l'algoritmo **Token Bucket**, utilizzando la chiave dell'utente (es. `request_rate_limiter.{fabio.auriemma}`) per verificare la disponibilità di "gettoni" nel secchio virtuale associato.

3. Scenari di Ammissione e Blocco Il blocco `alt` mostra i due possibili esiti della verifica su Redis:

- **Scenario di Ammissione (Tokens Disponibili):** Se il secchio contiene gettoni sufficienti, Redis decrementa il contatore e restituisce `allowed: true`. Il Gateway procede quindi all'inoltro della richiesta verso il microservizio di backend di destinazione (es. T7/Jacoco). Il servizio elabora la richiesta e restituisce una risposta `200 OK`, che il Gateway gira all'utente.
- **Scenario di Blocco (Tokens Esauriti):** Se il secchio è vuoto a causa di un numero eccessivo di richieste precedenti, Redis restituisce `allowed: false`. In questo caso, il Gateway blocca immediatamente il flusso e restituisce al client una risposta HTTP **429 Too Many Requests**.

È fondamentale notare come, nello scenario di blocco, la richiesta **non raggiunge mai** il microservizio di backend (T7). Questo dimostra l'efficacia del Rate Limiter nel proteggere i servizi computazionalmente onerosi dal sovraccarico.

2.2.4 Analisi Dinamica: Pipeline di Osservabilità

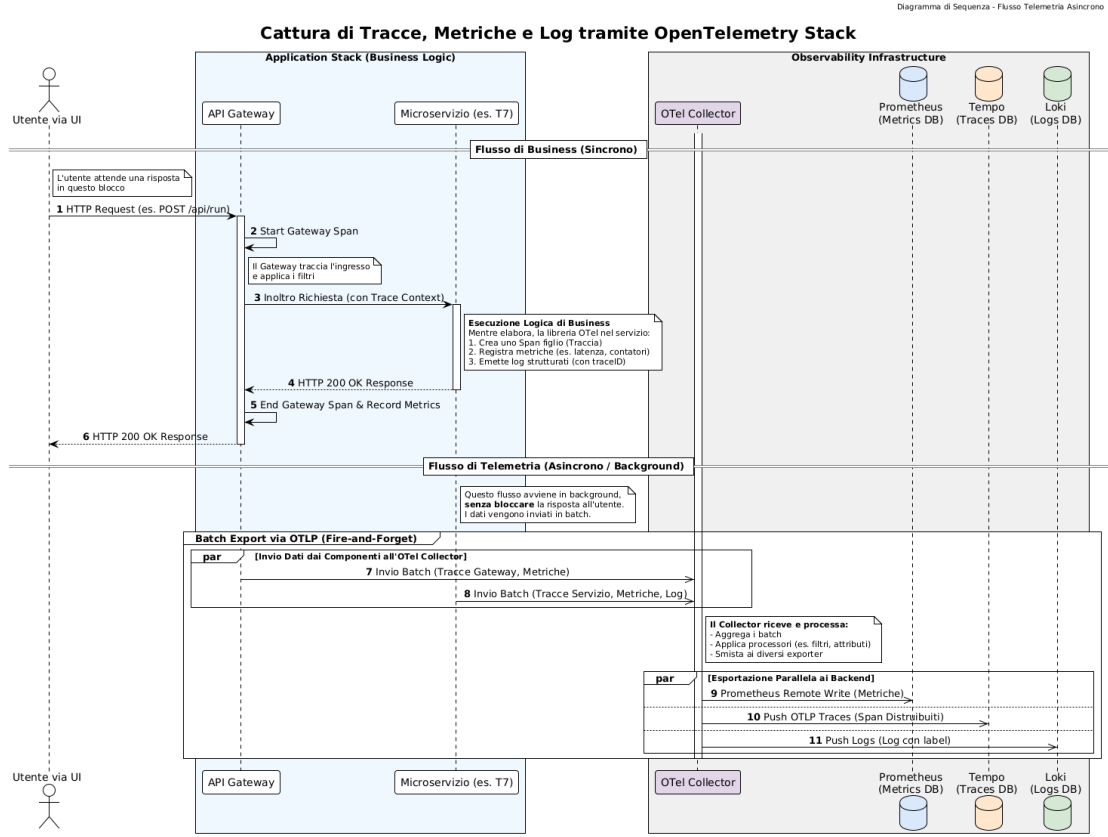


Figure 2.5: Sequence Diagram del flusso di telemetria.

2.3 Issue e Anti-pattern

È stata condotta un'analisi statica dell'architettura tramite lo strumento MARS [Tighilt et al., 2023]. Tale strumento è in grado di rilevare, attraverso l'applicazione di regole logiche ed euristiche, 16 anti-pattern. Le criticità emerse dall'indagine sono riassunte nelle tabelle seguenti, facendo riferimento alle definizioni fornite da [Cerny et al., 2023].

La Tabella 2.1 mostra gli anti-pattern per i quali è stata implementata

una soluzione diretta nel corso del progetto.

Anti-Pattern	Descrizione	Dove rilevato	Soluzione proposta
Local Logging	I log sono scritti sul file system locale, rendendo difficile la diagnosi e la correlazione tra servizi.	System Wide	Implementazione dello stack di osservabilità per centralizzare gestione dei log.
Insufficient Monitoring	Assenza di raccolta automatica di metriche (latenza, errori), impedendo il rilevamento proattivo dei guasti.	System Wide	Strumentazione del codice e dashboard di visualizzazione.
On-line Only	Uso di comunicazioni sincrone bloccanti per operazioni lunghe, con rischio timeout.	Interazioni T5 → T7/T8	Implementazione del Circuit Breaker per evitare attese lunghe in caso di malfunzionamenti.

Table 2.1: Riepilogo Issue e Anti-pattern risolti

La Tabella 2.2 elenca invece gli ulteriori anti-pattern rilevati dall'analisi statica per i quali, in questa fase del lavoro, non è stato applicato un refactoring correttivo, indicando tuttavia una possibile soluzione architetturale.

Anti-Pattern	Descrizione	Dove rilevato	Possibile soluzione
No API Versioning	Assenza di un meccanismo per gestire versioni diverse delle API, rischiando rotture di compatibilità.	System Wide	Introduzione di URI versioning (es. /v1/resource) o header custom.
Hardcoded Endpoints	Gli indirizzi dei servizi sono scritti staticamente nel codice, riducendo la flessibilità.	apiGateway, T1, T5	Adozione di un sistema di Service Discovery (es. Eureka) per la risoluzione dinamica.
Timeout	Manca di timeout espliciti, con rischio di blocco indefinito delle risorse.	apiGateway, T5	Configurazione di timeout specifici sui client HTTP e sui database.
Manual Configuration	Configurazioni gestite manualmente tramite file locali interni ai pacchetti di deploy.	System Wide	Centralizzazione tramite Spring Cloud Config Server per gestione esterna e dinamica.
No CI/CD	Assenza di pipeline automatizzate per build, test e deploy.	System Wide	Implementazione di workflow (es. GitHub Actions) per Continuous Integration.

Table 2.2: Altri Anti-pattern rilevati (Soluzioni future)

Chapter 3

Implementazione e Struttura del Progetto

3.1 Moduli Aggiunti o Modificati

Di seguito viene riportato il class diagram aggiornato dell'API Gateway e la configurazione aggiornata dei filtri.

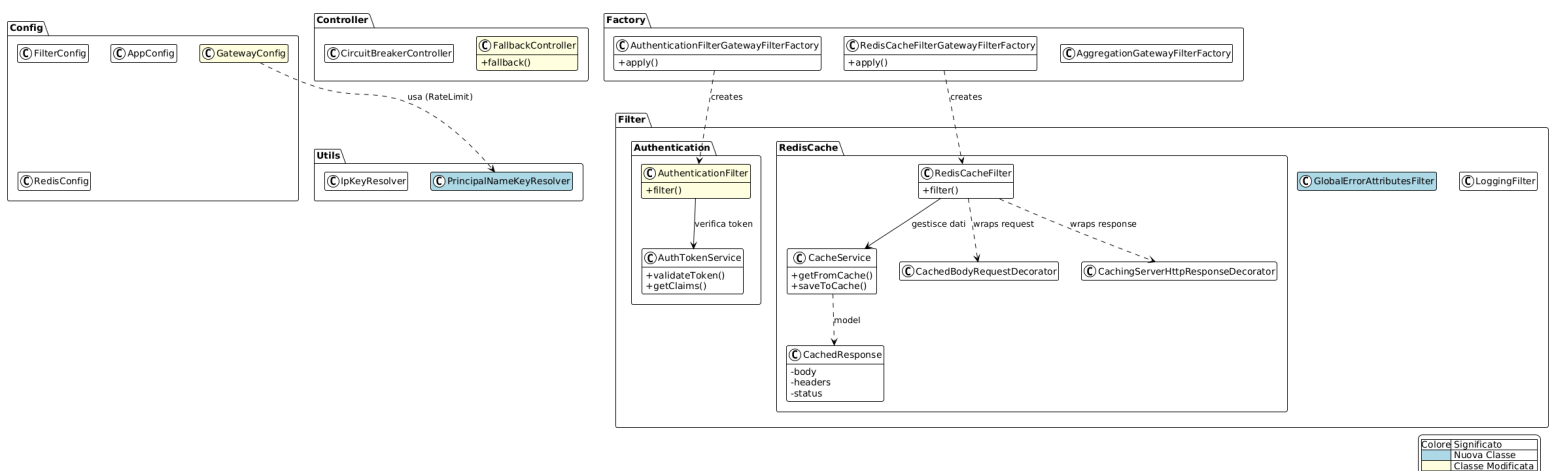


Figure 3.1: Class Diagram API-Gateway

Per mantenere il diagramma leggibile, la figura precedente si limita a mostrare le classi introdotte o modificate, omettendo la complessità delle dipendenze tipiche del framework Spring. Di seguito, verranno invece analizzate nel dettaglio le relazioni specifiche del meccanismo di configurazione dei filtri. Tale approfondimento permette di descrivere il contributo del task svolto, rappresentato dall'introduzione del **Global Filter** e dall'integrazione dei meccanismi di resilienza.

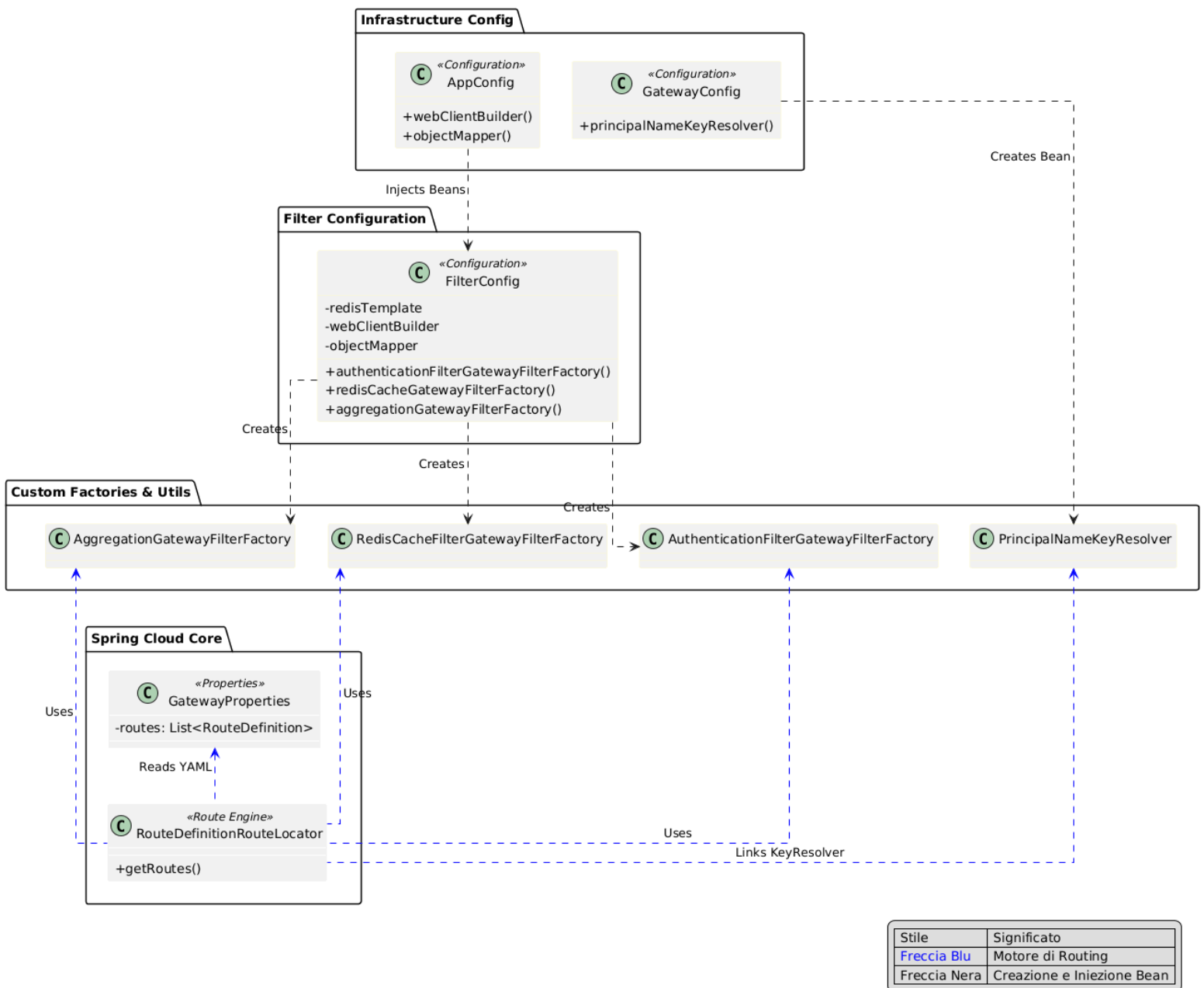


Figure 3.1: Configurazione dei Bean e Setup dei Filtri

Figure 3.2: Class Diagram API-Gateway con Spring Core

Gestione delle Dipendenze: La classe `AppConfig` inizializza i bean infrastrutturali (`WebClient`, `ObjectMapper`) e li **inietta** in `FilterConfig`. Questo applica il principio di *Inversion of Control*: le dipendenze non vengono create internamente, ma fornite pronte all'uso.

Pattern Factory: `FilterConfig` istanzia e registra nel contesto Spring

le Factory personalizzate (es. `AuthenticationFilterGatewayFilterFactory`

Questo permette al framework di avere a disposizione i "costruttori" necessari per i filtri definiti nel file YAML.

Motore di Routing: Il componente core `RouteDefinitionRouteLocator`

legge le configurazioni da `application.yml` e utilizza le Factory registrate in precedenza per tradurre le definizioni testuali delle rotte in filtri Java eseguibili.

Setup Rate Limiter: `GatewayConfig` espone il bean

`PrincipalNameKeyResolver`, che viene collegato al motore di routing per permettere al *Rate Limiter* di identificare gli utenti e conteggiare le richieste correttamente.

3.2 Implementazione del Circuit Breaker

In questa sezione vengono descritti i dettagli tecnici dell'implementazione del pattern *Circuit Breaker* all'interno dell'architettura a microservizi.

L'intervento si è focalizzato sull'API Gateway, che funge da punto d'ingresso e orchestrazione per le richieste verso i microservizi di backend, in particolare quelli identificati come critici: **T7** (Student Test Runner) e **T8** (EvoSuite Runner).

L'implementazione ha richiesto modifiche mirate alla configurazione delle dipendenze e alla definizione delle rotte e delle policy di resilienza.

3.2.1 Gestione delle Dipendenze

Il primo passo è stato l'integrazione della libreria necessaria per abilitare le funzionalità di Circuit Breaker in un ambiente basato su Spring Cloud Gateway, che utilizza un modello di programmazione reattivo (non bloccante) basato su Project Reactor.

```
1 <dependency>
2   <groupId>org.springframework.cloud</groupId>
3   <artifactId>spring-cloud-starter-circuitbreaker-reactor-
4     resilience4j</artifactId>
5 </dependency>
```

Listing 3.1: Presenza dipendenza Resilience4j nel pom.xml

Questa dipendenza fornisce le astrazioni necessarie e l'implementazione basata su Resilience4j, permettendo di gestire lo stato del circuito e i timeout in modo asincrono, preservando le performance del Gateway.

3.2.2 Configurazione del Routing

Il cuore dell'implementazione risiede nel file di configurazione `application.yml` dell'API Gateway. È stato necessario intervenire sulla definizione delle rotte per applicare il filtro `CircuitBreaker` specificamente ai servizi target.

Per le rotte `T7-route` e `T8-route`, è stato aggiunto il filtro nella catena di elaborazione, come mostrato di seguito:

```
1 spring:
```

```
2  cloud:
3    gateway:
4      routes:
5        - id: T7-route
6          uri: http://t7-controller:8087
7          predicates:
8            - Path=/compile/jacoco/**
9          filters:
10            - RewritePath=/compile/jacoco/(?<segment>.*), /${segment}
11              # ... altri filtri (Auth, RateLimiter) ...
12            - name: CircuitBreaker
13              args:
14                name: t7CircuitBreaker
15                fallbackUri: forward:/fallback/t7
16
17        - id: T8-route
18          uri: http://t8-controller:8088
19          predicates:
20            - Path=/compile/evosuite/**
21          filters:
22            - RewritePath=/compile/evosuite/(?<segment>.*),
23              /${segment}
24              # ... altri filtri ...
25            - name: CircuitBreaker
26              args:
27                name: t8CircuitBreaker
28                fallbackUri: forward:/fallback/t8
```

Listing 3.2: Applicazione del filtro CircuitBreaker nel file `application.yml`

La proprietà **name** nell'argomento del filtro (es. `t7CircuitBreaker`) è fondamentale, in quanto funge da chiave per collegare la rotta alla specifica configurazione di resilienza definita nella sezione successiva.

3.2.3 Configurazione dei Parametri di Resilienza

Sempre all'interno del file `application.yml`, è stata aggiunta una sezione dedicata (`resilience4j`) per definire il comportamento puntuale delle istanze di Circuit Breaker dichiarate.

I parametri sono stati calibrati considerando la natura "pesante" e potenzialmente lenta delle operazioni di compilazione e testing svolte da T7 e T8.

```
1 resilience4j:
2   circuitbreaker:
3     instances:
4       t7CircuitBreaker:
5         slidingWindowSize: 10
6         minimumNumberOfCalls: 5
7         permittedNumberOfCallsInHalfOpenState: 3
8         automaticTransitionFromOpenToHalfOpenEnabled: true
9         waitDurationInOpenState: 10s
10        failureRateThreshold: 50
11        eventConsumerBufferSize: 10
```

```
12     t8CircuitBreaker:
13         # Configurazione analoga per T8 con parametri specifici
14         slidingWindowSize: 10
15         minimumNumberOfCalls: 5
16         permittedNumberOfCallsInHalfOpenState: 2
17         automaticTransitionFromOpenToHalfOpenEnabled: true
18         waitDurationInOpenState: 20s
19         failureRateThreshold: 50
20
21     timelimiter:
22         instances:
23             t7CircuitBreaker:
24                 timeoutDuration: 180s
25             t8CircuitBreaker:
26                 timeoutDuration: 300s
```

Listing 3.3: Configurazione parametri Resilience4j in application.yml

Analisi dei Parametri

Di seguito il significato delle configurazioni adottate:

- **slidingWindowSize: 10:** Il Circuit Breaker utilizza una finestra scorrevole basata sul conteggio per memorizzare l'esito delle ultime 10 chiamate.
- **minimumNumberOfCalls: 5:** È il numero minimo di chiamate che devono essere registrate nella finestra prima che il Circuit

Breaker possa calcolare il tasso di errore e decidere se aprire il circuito.

- **failureRateThreshold: 50:** Se il 50% delle chiamate nella finestra fallisce (es. 5 su 10), lo stato passa a *OPEN*.
- **waitDurationInOpenState: 10s:** Definisce quanto tempo il circuito rimane aperto prima di passare allo stato *HALF-OPEN* e tentare di inviare richieste di prova.
- **permittedNumberOfCallsInHalfOpenState: 3:** Quando il circuito è in fase di test (*HALF-OPEN*), permette il passaggio di sole 3 richieste per verificare se il servizio backend si è ripreso.
- **timeoutDuration (TimeLimiter):** Impostato a 180s per T7 e 300s per T8. Questo parametro è cruciale: definisce il tempo massimo che il Gateway attende per una risposta dal microservizio prima di considerare la chiamata fallita (timeout) e attivare il fallback. Valori elevati sono stati scelti per accomodare i tempi di esecuzione intrinseci dei processi di build e test.

Questa configurazione garantisce che il sistema sia reattivo ai guasti (aprendo il circuito velocemente in caso di errori ripetuti) ma sufficientemente paziente con le operazioni legittimamente lunghe (grazie al TimeLimiter esteso).

3.3 Ottimizzazione e Configurazione di EvoSuite

L'intervento di rifattorizzazione si è concentrato principalmente sulla riconfigurazione del wrapper Java che gestisce l'esecuzione di EvoSuite, al fine di massimizzare l'efficienza nell'utilizzo delle risorse hardware del server (dotato di 32GB di RAM) e ridurre i tempi di latenza. Le modifiche implementative hanno coinvolto due componenti chiave del microservizio: la classe di servizio `CoverageService.java` e la classe di utilità `BuildResponse.java`.

3.3.1 Configurazione della JVM e Parametri di EvoSuite

La modifica più impattante ha riguardato la parametrizzazione della Java Virtual Machine (JVM) invocata tramite `ProcessBuilder` all'interno di `CoverageService.java`. Analizzando il comportamento del container, è emerso che la configurazione di default causava frequenti interruzioni per la gestione della memoria (*Garbage Collection*), limitando il throughput di generazione dei test.

Per ovviare a questo problema, è stata adottata una strategia *High-Memory Sequential*, definendo esplicitamente i seguenti parametri di avvio per il processo EvoSuite:

- **Allocazione Heap (`-Xmx6144m`):** È stato incrementato il limite

massimo della memoria Heap a 6GB. Questa allocazione, calibrata sulle risorse del container ospite, permette a EvoSuite di mantenere in memoria l'intero grafo degli oggetti e delle mutazioni genetiche senza dover ricorrere allo swapping o a frequenti cicli di pulizia, eliminando di fatto i colli di bottiglia legati alla memoria.

- **Garbage Collector (-XX:+UseParallelGC):** È stato imposto l'utilizzo del *Parallel Garbage Collector*. Questo algoritmo è ottimizzato per il *throughput*, sfruttando tutti i core della CPU per eseguire le operazioni di pulizia della memoria nel minor tempo possibile durante le fasi di calcolo intensivo.

Il comando di esecuzione risultante, implementato nel metodo `calculateEvosuiteCoverage`, si presenta come segue:

Listing 3.4: Configurazione ottimizzata del processo EvoSuite

```
runCommand(workingDir, 5,  
    "/usr/lib/jvm/java-8-openjdk-amd64/bin/java",  
    "-Xmx6144m",           // Allocazione 6GB RAM  
    "-XX:+UseParallelGC", // Throughput Collector  
    "-jar", workingDir + "/evosuite-1.0.6.jar",  
    "-measureCoverage",  
    // ... altri parametri ...  
    "-Dcriterion=" + criterion  
);
```

3.3.2 Processo di Build Maven

Parallelamente al tuning della JVM, è stata rivista la logica di invocazione di Maven. Il codice originale eseguiva una sequenza frammentata di comandi (**clean**, **install**, **dependency**), causando l'avvio multiplo della JVM. L'implementazione è stata modificata per eseguire un unico comando atomico:

```
mvn package dependency:copy-dependencies
```

Il comando **clean** serve a cancellare i file vecchi prima di ricompilare. Tuttavia, il nostro sistema crea una cartella nuova e vuota per ogni partita, quindi è stata rimossa.

3.3.3 Adattamento del Data Transfer Object

Infine, è stata modificata la classe **BuildResponse.java**. Poiché sono state rimosse dall'esecuzione le metriche non visualizzate dall'interfaccia utente (**Output**, **Method**, **MethodNoException**), è stato necessario aggiornare la logica di parsing del file CSV dei risultati, adattando gli indici di lettura per mappare correttamente i valori rimanenti nel DTO di risposta (**EvosuiteScoreDTO**).

3.4 Implementazione del Rate Limiter

In questa sezione vengono descritti i dettagli tecnici dell'implementazione del pattern Rate Limiter all'interno dell'API Gateway. L'obiettivo

è proteggere i microservizi critici (T7 e T8) da picchi di traffico e abusi, garantendo che il carico rimanga entro i limiti operativi definiti. L'architettura scelta si basa su **Redis**, che funge da store distribuito per la gestione dei token (algoritmo Token Bucket), permettendo di mantenere lo stato delle richieste anche in un ambiente con repliche multiple del Gateway.

3.4.1 Configurazione del Key Resolver

A differenza di un Rate Limiter basato sul semplice indirizzo IP, l'implementazione realizzata mira a limitare le richieste in base all'identità dell'utente autenticato. Per ottenere questo comportamento, è stata creata una classe di configurazione `GatewayConfig.java` e implementato un `KeyResolver` personalizzato.

Il bean `PrincipalNameKeyResolver` estrae il *Principal* (il nome utente) direttamente dal token JWT o dalla sessione di sicurezza.

Listing 3.5: Configurazione del `PrincipalNameKeyResolver`

```
@Bean
@Primary
public KeyResolver principalNameKeyResolver() {
    return exchange -> exchange.getPrincipal()
        .flatMap(principal -> Mono.just(principal.getName
            ()))
        .defaultIfEmpty("anonymous");
}
```

Questa configurazione assicura che il conteggio delle richieste sia legato allo specifico account utente, indipendentemente dalla macchina o dall'indirizzo IP da cui proviene la chiamata.

3.4.2 Configurazione del Routing e Parametri

La configurazione puntuale dei filtri di Rate Limiting è stata applicata nel file `application.yml`, all'interno della definizione delle rotte per T7 e T8. Il filtro `RequestRateLimiter` utilizza tre argomenti principali per definire il comportamento dell'algoritmo Token Bucket:

- `redis-rate-limiter.replenishRate`: Il numero di token generati al secondo (velocità di riempimento del secchio).
- `redis-rate-limiter.burstCapacity`: La capacità massima del secchio (picco di richieste consentito in un istante).
- `redis-rate-limiter.requestedTokens`: Il costo in token di una singola richiesta (impostato a 1).

Di seguito è riportata la configurazione adottata, differenziata in base al carico computazionale dei servizi:

Listing 3.6: Configurazione dei filtri Rate Limiter in `application.yml`

```
spring:
  cloud:
    gateway:
      routes:
```

```
- id: T7-route
  uri: http://t7-controller:8087
  predicates:
    - Path=/compile/jacoco/**
  filters:
    - RewritePath=/compile/jacoco/(?<segment>.*),
      /${segment}
    - name: RequestRateLimiter
      args:
        redis-rate-limiter.replenishRate: 2
        redis-rate-limiter.burstCapacity: 13
        redis-rate-limiter.requestedTokens: 1
        key-resolver: "#{
          @principalNameKeyResolver}"

- id: T8-route
  uri: http://t8-controller:8088
  predicates:
    - Path=/compile/evosuite/**
  filters:
    - RewritePath=/compile/evosuite/(?<segment
      >.*), /${segment}
    - name: RequestRateLimiter
      args:
        redis-rate-limiter.replenishRate: 1
        redis-rate-limiter.burstCapacity: 8
        redis-rate-limiter.requestedTokens: 1
        key-resolver: "#{
```

```
@principalNameKeyResolver}"
```

Analisi delle scelte configurative

- **T7 (Jacoco):** È stata scelta una configurazione più tollerante (`burstCapacity: 13`), poiché l'analisi statica è un'operazione a medio carico. Questo permette agli utenti un utilizzo fluido senza blocchi durante la normale navigazione.
- **T8 (EvoSuite):** È stata applicata una politica restrittiva (`burstCapacity: 8` e `replenishRate: 1`). Essendo la generazione di test un'operazione *CPU-intensive* che alloca molta memoria, limitare aggressivamente la concorrenza è necessario per prevenire il *Resource Exhaustion* del container.

3.5 Configurazione Prometheus e Otel

Di seguito sono stati riportati i passaggi relativi all'implementazione dell'osservabilità LGTM per un servizio generico.

3.5.1 Dipendenze necessarie

```
<!-- Micrometer Prometheus registry -->  
<dependency>  
  <groupId>io.micrometer</groupId>
```

```
<artifactId>micrometer-registry-prometheus</  
    artifactId>  
</dependency>  
<!-- Logback JSON encoder -->  
<dependency>  
    <groupId>net.logstash.logback</groupId>  
    <artifactId>logstash-logback-encoder</artifactId>  
    <version>6.6</version>  
</dependency>  
<!-- OpenTelemetry Spring Boot -->  
<dependency>  
    <groupId>io.opentelemetry.instrumentation</groupId>  
    <artifactId>opentelemetry-spring-boot-starter</  
        artifactId>  
    <version>2.7.0</version>  
</dependency>
```

- **micrometer-registry-prometheus:** Agisce da traduttore. Converte i dati interni di Spring nel formato che Prometheus capisce ed espone l'endpoint `/actuator/prometheus` necessario per la raccolta dati.
- **logstash-logback-encoder:** Serve a strutturare i log. Trasforma le righe di testo in oggetti JSON, permettendo a Loki di indicizzare i campi e fare ricerche precise (es. filtrare solo gli errori).
- **opentelemetry-spring-boot-starter:** Gestisce le tracce. Intercepta il traffico per inviarlo a Tempo e inietta il `trace_id`

nei log, creando il collegamento fondamentale tra il mondo del Tracing e quello del Logging.

3.5.2 File .properties

```
server.port=[PORTO_SERVIZIO]

# application.properties
variabile.mvn=mvn
logging.level.root=WARN
logging.level.RemoteCCC.App.config=INFO

#Attivazione prometheus
management.endpoints.web.exposure.include=health,info,
    prometheus
management.endpoint.prometheus.enabled=true
management.metrics.export.prometheus.enabled=true
management.endpoints.web.base-path=/actuator

# Configurazione OpenTelemetry con OpenTelemetry Protocol
    e OpenTelemetry Collector
otel.resource.attributes.service.name=[NOME_SERVIZIO]
otel.traces.exporter=otlp
otel.metrics.exporter=otlp
otel.exporter.otlp.traces.endpoint=http://otel-collector
    :4317
otel.exporter.otlp.protocol=grpc
```

Si nota una parziale ridondanza nell'esportazione delle metriche, gius-

tificata dall'adozione di un approccio ibrido. Mentre Micrometer è indispensabile per ottenere metriche profonde a livello applicativo e di framework, OpenTelemetry assume qui un ruolo cruciale per la gestione delle Tracce Distribuite e per garantire la correlazione automatica tra queste e i log, offrendo inoltre una standardizzazione delle metriche infrastrutturali verso il Collector.

3.5.3 File resources/logback-spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <include resource="org/springframework/boot/logging/
    logback/defaults.xml"/>
  <appender name="CONSOLE" class="ch.qos.logback.core.
    ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.
      LoggingEventCompositeJsonEncoder">
      <providers>
        <timestamp/>
        <logLevel/>
        <loggerName/>
        <threadName/>
        <message/>
        <stackTrace/>
        <mdc>
          <includeMdcKeyName>trace_id</
          includeMdcKeyName>
```

```
        includeMdcKeyName>span_id</
        includeMdcKeyName>
    </mdc>
</providers>
</encoder>
</appender>
<root level="INFO">
    <appender-ref ref="CONSOLE"/>
</root>
</configuration>
```

Per garantire una forma specifica ai log generati è stato sufficiente aggiungere questo file all'interno della cartella resources del progetto.

3.5.4 File docker-compose.yml

```
name: [SERVIZIO]
services:
  app:
    image: [IMMAGINE_SERVIZIO]
    container_name: [NOME_CONTAINER]
    restart: always
    environment:
      OTEL_EXPORTER_OTLP_ENDPOINT: "http://otel-collector
        :4317"
      OTEL_EXPORTER_OTLP_PROTOCOL: grpc
    expose:
      - [PORTO_SERVIZIO]
    networks:
```



```
- global-network
networks:
  global-network:
    external: true
```

Per rendere nota al servizio la porta su cui è necessario inviare i dati raccolti tramite otel collector, vanno impostate delle variabili d'ambiente che indicano il protocollo con cui tali informazioni viaggiano e la porta che devono raggiungere.

3.5.5 Configurazione prometheus_config.yml

```
- job_name: 'microservice-X'
  metrics_path: /actuator/prometheus
  static_configs:
    - targets: ['[NOME_CONTAINER]:[PORTO_CONTAINER]']
      labels:
        service: '[NOME_SERVIZIO]'
```

L'ultimo passo per ultimare la configurazione del servizio con prometheus è aggiungere la configurazione del servizio all'interno del file `prometheus_config.yml` situato in `/observability/prometheus/`

3.6 Configurazione e Verifica di Grafana

All'interno dello stack LGTM, **Grafana** ricopre il ruolo cruciale di piattaforma di visualizzazione e analisi unificata. Esso agisce come punto di accesso centrale, permettendo di aggregare e correlare visivamente i tre pilastri dell'osservabilità: le metriche raccolte da Prometheus, i log indicizzati da Loki e le tracce distribuite gestite da Tempo.

A seguito dell'esecuzione degli script **build.sh** e **deploy.sh**, i container relativi allo stack di osservabilità risulteranno attivi. Di seguito sono riportati i passaggi per la verifica delle fonti dati e l'importazione delle dashboard.

3.6.1 Verifica dei Data Sources

1. Accedere all'interfaccia web di Grafana all'indirizzo **http://localhost:3000**.
2. Navigare nella sezione **Data Sources** (solitamente sotto *Connections* o *Configuration*).
3. Verificare che siano presenti e correttamente configurate le tre fonti dati principali: **Tempo**, **Prometheus** e **Loki**.
4. *Nota di troubleshooting*: Nel caso in cui le fonti dati non fossero visibili, forzare il riavvio dello stack eseguendo il comando **docker-compose up** all'interno della directory **observability**.

3.6.2 Importazione delle Dashboard

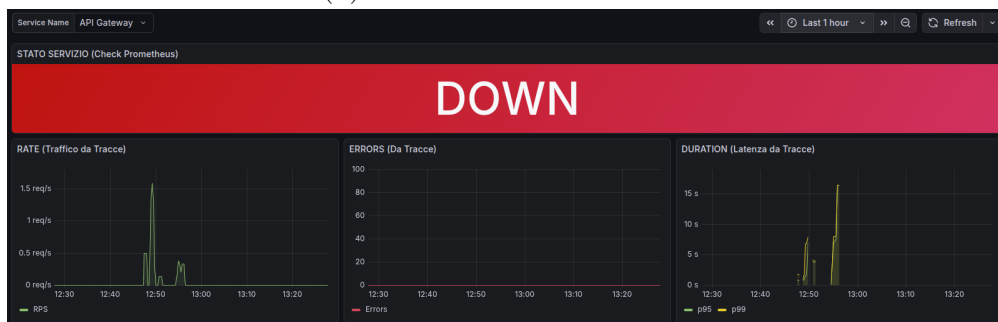
Per agevolare l'analisi dinamica delle metriche, sono state predisposte delle dashboard preconfigurate. La procedura di importazione è la seguente:

1. Nel menu laterale di Grafana, selezionare **Dashboards** → **New** → **Import**.
2. Recuperare il codice JSON da uno dei file situati nella directory del progetto: `/observability/dashboard/`.
3. Copiare il contenuto del file JSON e incollarlo nell'apposito pannello di importazione su Grafana.
4. Confermare l'operazione per visualizzare i grafici.

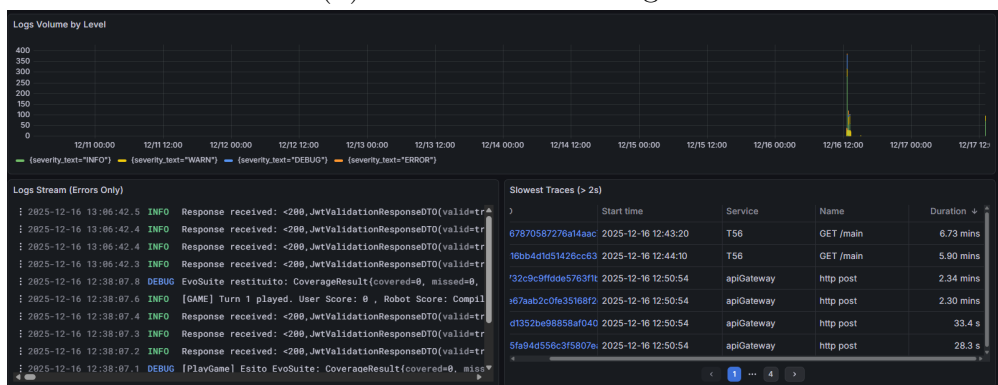
CHAPTER 3. IMPLEMENTAZIONE E STRUTTURA DEL PROGETTO



(a) Dashboard Panoramica



(b) Dashboard di Dettaglio



(c) Dashboard Logs e Slow Traces

Figure 3.3: Panoramica delle tre dashboard di monitoraggio configurate su Grafana.

Chapter 4

Deployment

4.1 Deployment Diagram

La struttura dell'architettura generale è rimasta invariata eccetto per l'aggiunta del servizio Prometheus nello stack di osservabilità.

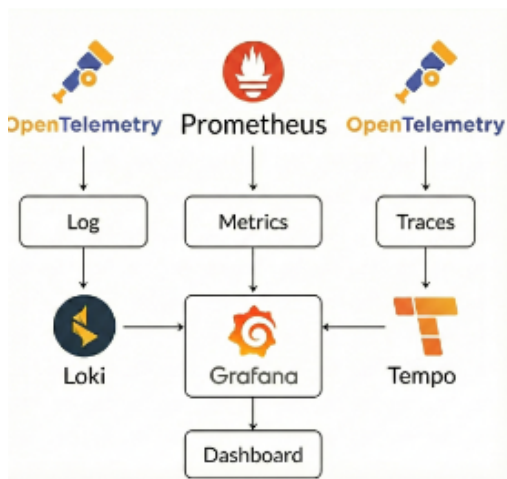


Figure 3.7: Stack di osservabilità

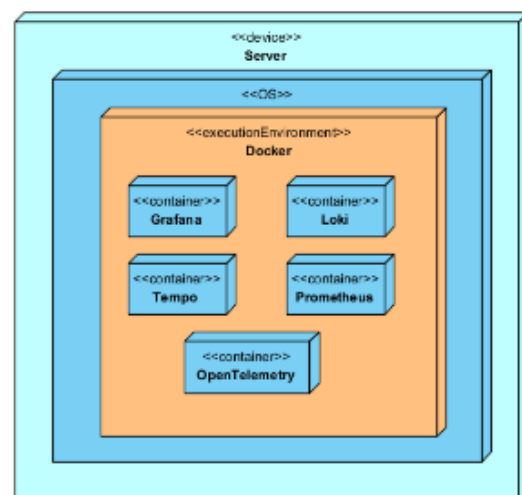


Figure 3.8: Deploy dello stack

Chapter 5

Testing

5.1 Test Effettuati

In questo capitolo vengono descritte le attività di verifica e validazione condotte sui meccanismi di resilienza implementati. La strategia di testing ha seguito un approccio bottom-up, partendo dalla validazione isolata dei singoli filtri implementati per arrivare alla verifica del sistema integrato in scenari di carico reale. La prima fase di testing si è concentrata sulla validazione funzionale dei singoli pattern di resilienza, *Rate Limiter* e *Circuit Breaker*, in un ambiente isolato. L'obiettivo primario è stato garantire che la configurazione di base rispondesse correttamente alle specifiche definite.

5.1.1 Validazione del Rate Limiter

I test sul Rate Limiter sono stati eseguiti per verificare la capacità del componente di limitare il flusso di richieste in ingresso secondo le soglie preimpostate nel file di configurazione (**replenishRate:2, burstCapacity:10** per T7 e **replenishRate:2,burstCapacity:15** per T8). È stato simulato un traffico superiore al limite consentito per osservare il comportamento di rifiuto delle richieste in eccesso.

Utilizzando **Postman Collection Runner**, sono state lanciate 14 richieste consecutive senza delay verso l'endpoint `/compile/jacoco/`.

- **Esito T7:** Le prime 14 richieste (entro la capacità di burst) vengono accettate (HTTP 200), mentre le successive vengono respinte con lo status **HTTP 429 Too Many Requests**, per poi riprendere una volta ricaricati i token confermando l'efficacia del filtro basato su Redis.

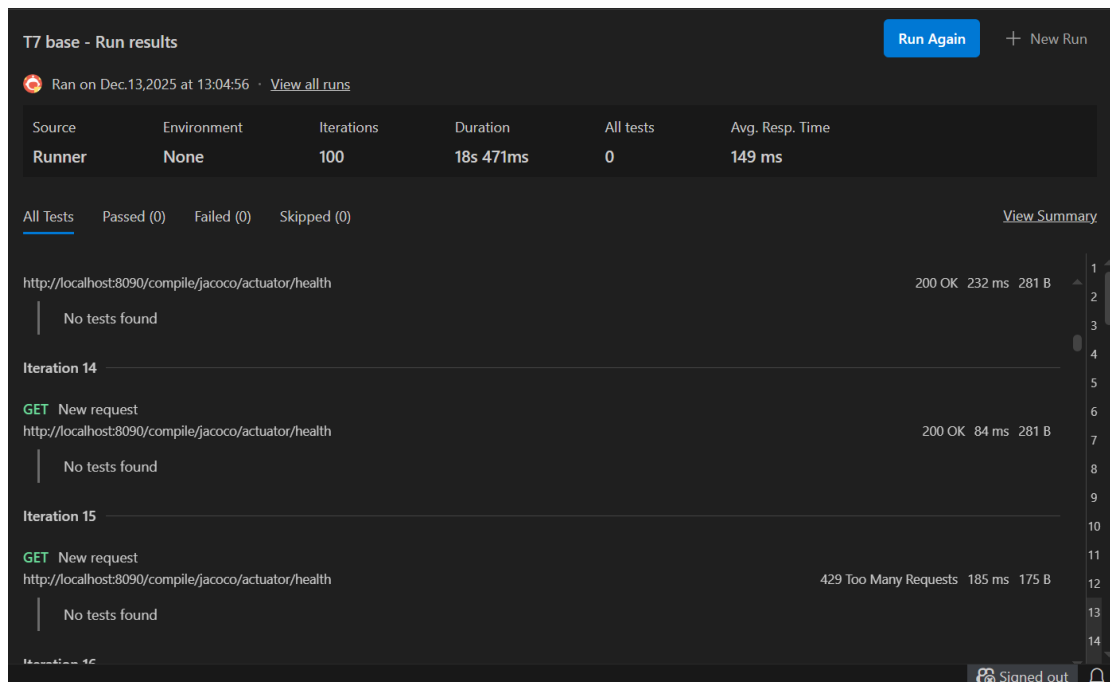


Figure 5.1: Test con parametri base per T7

- **Tuning T7:** Essendo T7 un servizio di analisi statica con un carico computazionale medio, è stata privilegiata la fluidità dell'esperienza utente. I parametri sono stati fissati a **replenishRate: 2** e **burstCapacity: 13**. I test hanno dimostrato che questa configurazione permette all'utente di effettuare fino a 40 richieste consecutive prima di essere bloccato. Questo approccio rende il Rate Limiter "invisibile" durante il normale utilizzo, intervenendo solo in casi di anomalie tecniche (es. loop infiniti) o abusi estremi.

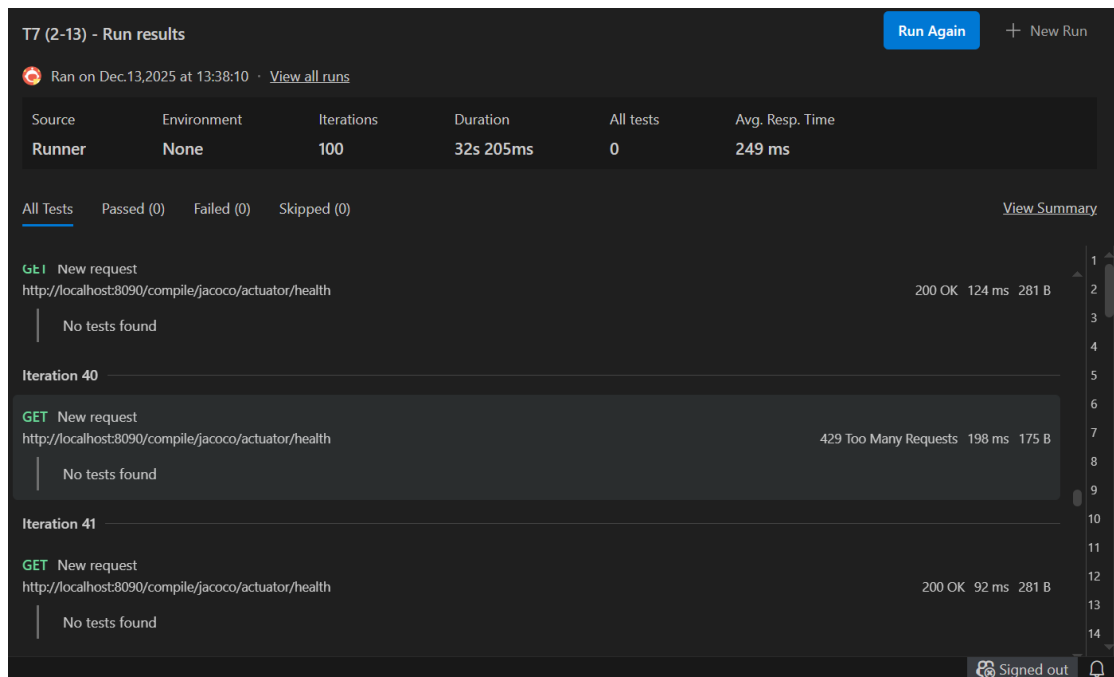


Figure 5.2: Test per T7 con nuovi parametri

- **Esito T8:** Le prime 25 richieste vengono accettate (HTTP 200), mentre le successive vengono respinte con lo status **HTTP 429 Too Many Requests**.

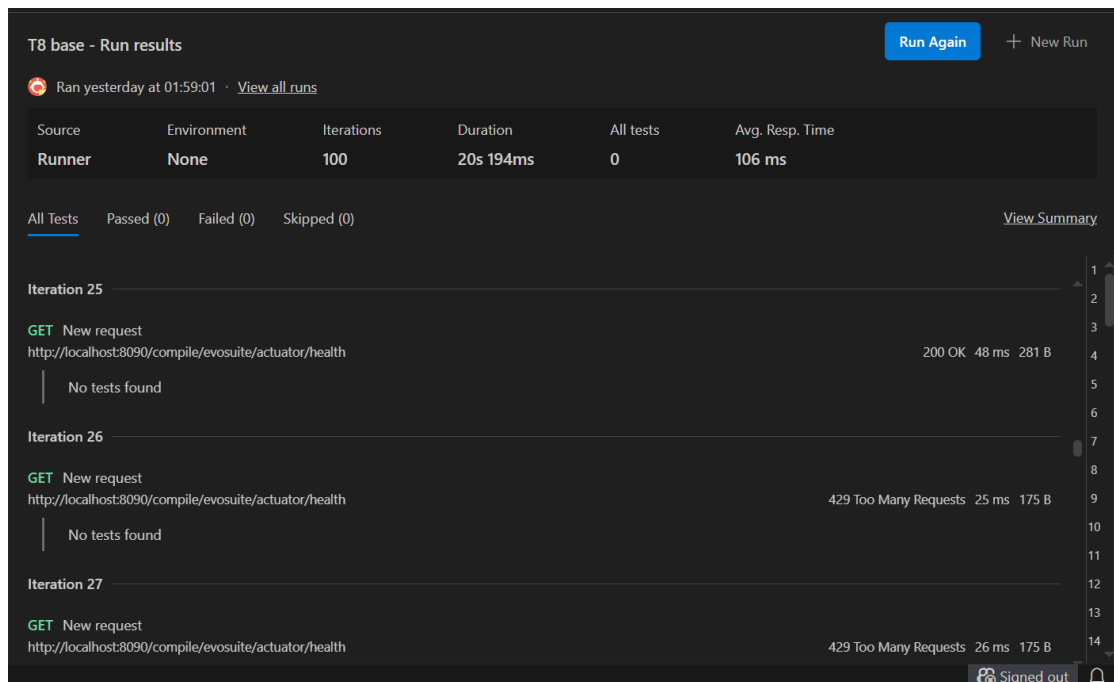


Figure 5.3: Test con parametri base per T8

- **Tuning T8:** Per T8, che esegue algoritmi genetici per la generazione di test (operazione *CPU-intensive* e dispendiosa in termini di memoria), è stata adottata una politica restrittiva. I parametri sono stati impostati a **replenishRate: 1** e **burstCapacity: 8**. I test hanno confermato che il blocco scatta già alla 14esima richiesta consecutiva in un intervallo breve.

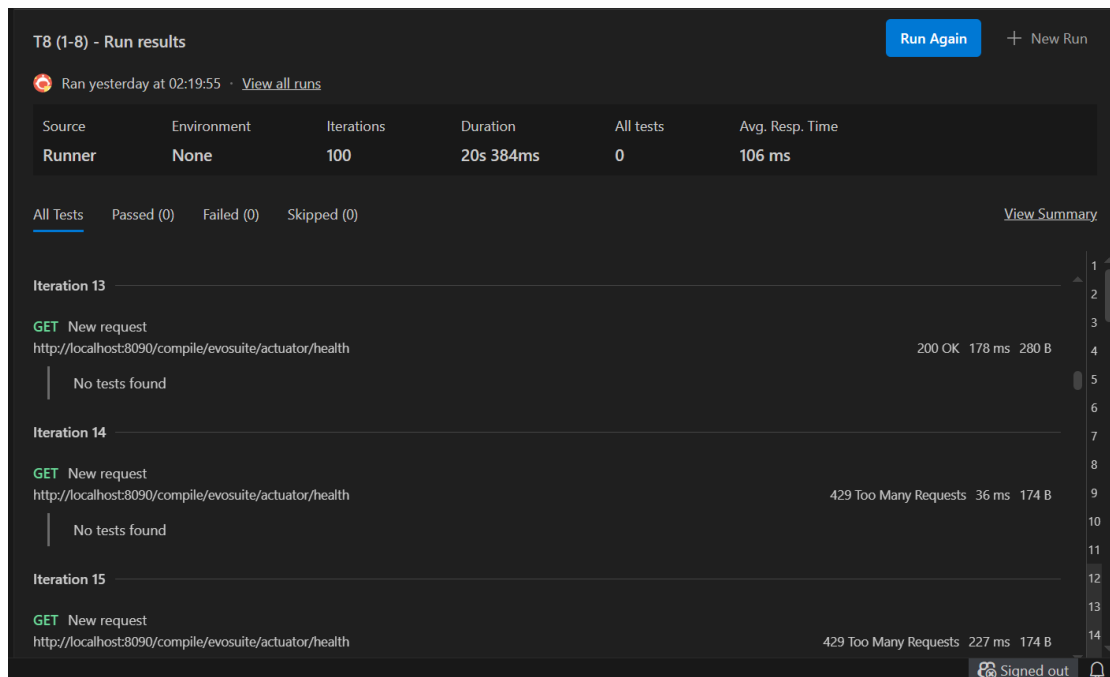


Figure 5.4: Test per T8 con nuovi parametri

5.1.2 Validazione del CircuitBreaker

Parallelamente, è stata verificata la macchina a stati del Circuit Breaker. I test hanno simulato malfunzionamenti nel servizio di backend T7 (arresto del container Docker) per forzare l'apertura del circuito e verificare la successiva transizione allo stato *Half-Open* dopo il periodo di attesa configurato di 10 secondi.

Sono stati eseguiti due scenari principali tramite Postman:

1. **Scenario di Guasto (Fault Injection):** Con il servizio T7 arrestato, sono state inviate 10 richieste.
 - *Fase Accumulo:* Le prime 5 richieste hanno mostrato latenze elevate (timeout), confermando lo stato *Closed*.

- *Fase Protezione:* Dalla sesta richiesta, il Circuit Breaker è passato allo stato *Open*, restituendo immediatamente il JSON di fallback con status **HTTP 503 Service Unavailable** e tempi di risposta $< 20\text{ms}$.

2. **Scenario di Ripristino:** Dopo aver riavviato T7 e atteso il timeout di reset, le richieste verso l'endpoint di health check (`/actuator/health`) hanno restituito **HTTP 200 OK**, confermando la chiusura del circuito.

Table 5.1: Riepilogo Esecuzione Test Circuit Breaker - Scenario Guasto

Iter.	Stato	Code	Tempo	Note
1	CLOSED	503	~5000	Timeout connessione
2	CLOSED	503	~800	Accumulo errori
3	CLOSED	503	~800	Accumulo errori
4	CLOSED	503	~800	Accumulo errori
5	CLOSED	503	~800	Soglia raggiunta
6	OPEN	503	10	Circuito Aperto
7	OPEN	503	8	Protezione attiva
8	OPEN	503	14	Protezione attiva
9	OPEN	503	10	Protezione attiva
10	OPEN	503	12	Protezione attiva

Table 5.2: Riepilogo Esecuzione Test Circuit Breaker - Scenario Ripristino

Iter.	Stato	Code	Tempo	Esito
1	CLOSED	200	19	Successo (Service UP)
2	CLOSED	200	13	Successo (Service UP)
3	CLOSED	200	16	Successo (Service UP)
4	CLOSED	200	17	Successo (Service UP)
5	CLOSED	200	17	Successo (Service UP)

5.1.3 Test di Integrazione

Una volta validati i singoli componenti, si è proceduto con i test di integrazione focalizzati sulla catena dei filtri del Gateway. In questa fase, l'obiettivo è stato verificare la corretta orchestrazione e sequenzialità dei pattern.

È stato verificato che i filtri applichino le logiche di controllo nell'ordine corretto definito dalla priorità (**Ordered**):

1. **Authentication Filter:** Verifica preliminare delle credenziali. Se fallisce (401), il Rate Limiter e il Circuit Breaker non vengono invocati, risparmiando risorse.
2. **Rate Limiter:** Interviene a monte del Circuit Breaker. Se il traffico è eccessivo (429), la richiesta viene bloccata prima di interrogare lo stato del circuito o il backend.
3. **Circuit Breaker:** Ultimo componente interpellato prima dell'inoltro al microservizio.

I test sono stati condotti utilizzando uno script python facente utilizzo della libreria **requests**. La struttura del progetto è la seguente:

- **api_gateway_test.py:** Script principale che orchestra l'esecuzione della suite di test. Contiene l'implementazione dei singoli Test Case.

- **auth_manager.py**: Modulo di utilità . Si occupa della gestione, caching e refresh del cookie di sessione (incluso il **jwt**). Ottiene il token facendo una POST con delle credenziali valide verso l'endpoint `localhost:8090/userService/auth/login"`
- **config.py**: File di configurazione che centralizza le costanti globali del progetto, quali gli URL degli endpoint del microservizio, i payload JSON di test standardizzati e i parametri di timeout per le richieste HTTP.
- **.env**: File di variabili d'ambiente utilizzato per archiviare dati sensibili e configurazioni locali come le credenziali di accesso per recuperare il token *jwt*.

I principali test sono:

- **TC-01: Flusso Nominale (Happy Path)**. Simula una richiesta standard inviata da un utente correttamente autenticato. Verifica che il sistema, in condizioni operative normali, elabori la richiesta e restituisca un esito positivo (HTTP 200).
- **TC-02: Autenticazione Fallita**. Invia una richiesta corredata da un token JWT invalido o assente. Il test ha lo scopo di validare i filtri di sicurezza, accertando che il Gateway blocchi l'accesso non autorizzato restituendo l'errore HTTP 401.
- **TC-03: Violazione Rate Limiter (Stress Test)**. Esegue richieste parallele per superare la soglia di traffico consentita.

Verifica che il meccanismo di *Token Bucket* intervenga bloccando le richieste in eccesso con codice HTTP 429 (Too Many Requests).

- **TC-04: Circuit Breaker Aperto.** Verifica il comportamento del sistema quando il microservizio di backend è irraggiungibile (al momento deve essere spento manualmente). Il test controlla che il filtro *Circuit Breaker* transiti nello stato *OPEN*, restituendo un errore di servizio non disponibile (HTTP 503).

Questi test oltre che essere utili per verificare il corretto funzionamento dei filtri quando questi sono tutti attivi, è anche utile per verificare che il messaggio di errore sia stato correttamente standardizzato.

```

=====
CONFIGURAZIONE ATTUALE
=====
Base URL: http://localhost:8090
Username: testeruno@gmail.com
Password: ✓ Configurata
JWT Cache: .jwt_cache.json

✓ Configurazione completa
=====

TEST SUITE API GATEWAY - MENU PRINCIPALE
=====

1. TC-01 - Flusso Nominale (Happy Path)
2. TC-02 - Autenticazione Fallita
3. TC-03 - Violazione Rate Limiter
4. TC-04 - Circuit Breaker Aperto

Opzioni Speciali:
0. Esegui TUTTI i test (automatico: TC-01, TC-02, TC-03)
a. Informazioni autenticazione
c. Pulisci cache JWT
q. Esci

Seleziona un'opzione:

```

(a) Menu e Configurazione

```

=====
ESECUZIONE AUTOMATICA TEST
=====

Esecuzione TC-01...

TEST TC-01: Flusso Nominale (Happy Path)
=====
Token caricati dalla cache (salvati: 2025-12-17T14:03:21.258432)
✓ JWT valido trovato in cache
Token Info: User: testeruno@gmail.com | Role: PLAYER | Scade: 2025-12-17 15:03:21 | Rimangono: 57h 58s
Status Code: 200 (Atteso: 200)
JSON Body: Ignorato o vuoto nello schema atteso.
✓ TEST SUPERATO (Basato solo su Status Code)

Esecuzione TC-02...

```

(b) Esecuzione TC-01 (Happy Path)

```

=====
TEST TC-02: Autenticazione Fallita
=====
Invio richiesta con token invalido...
Status Code: 401 (Atteso: 401)

--- Analisi Campi JSON ---
✓ type: about:blank
✓ title: Unauthorized
✓ status: 401
✓ detail: Autenticazione mancante o non valida.
✓ instance: /compile/jacoco/coverage/player

✓ TEST SUPERATO (Basato solo su Status Code)

Esecuzione TC-03...

TEST TC-03: Violazione Rate Limiter (Stress Test)
=====
Avvio (50 thread, max 150 richieste)...
Token caricati dalla cache (salvati: 2025-12-17T14:03:21.258432)
✓ JWT valido trovato in cache
Token Info: User: testeruno@gmail.com | Role: PLAYER | Scade: 2025-12-17 15:03:21 | Rimangono: 57h 51s

[PASSATO] Status: 429 | Rate Limit Intercettato correttamente.
Statistiche Status Code: {429: 1}
Status Code: 429 (Atteso: 429)

--- Analisi Campi JSON ---
✓ type: about:blank
✓ title: Too Many Requests
✓ status: 429
✓ detail: Il limite di richieste è stato superato. Riprova più tardi.
✓ instance: /compile/jacoco/coverage/player

✓ TEST SUPERATO (Basato solo su Status Code)

RIEPILOGO TEST
=====
TC-01: ✓ PASSED
TC-02: ✓ PASSED
TC-03: ✓ PASSED

```

(c) Esecuzione TC-02 e TC-03

```

=====
TEST TC-04: Circuit Breaker - Aperto
=====
⚠ Requisito: BACKEND deve essere DOWN
Prova: Invio quando il backend è spento...
Token caricati dalla cache (salvati: 2025-12-17T14:03:21.258432)
✓ JWT valido trovato in cache
Token Info: User: testeruno@gmail.com | Role: PLAYER | Scade: 2025-12-17 15:03:21 | Rimangono: 56h 48s
Status Code: 503 (Atteso: 503)

--- Analisi Campi JSON ---
✓ type: about:blank
✓ title: Service Unavailable
✓ status: 503
✓ detail: Il servizio di compilazione (??) è momentaneamente sovraccarico o non raggiungibile. Riprova tra qualche istante.
✓ instance: /libbwa/??

✓ TEST SUPERATO (Basato solo su Status Code)

```

(d) Esecuzione TC-04 (Circuit Breaker)

Figure 5.5: Output di esecuzione della Test Suite.

Chapter 6

Conclusioni

6.1 Sviluppi Futuri e Raccomandazioni

- **Estensione della copertura di monitoraggio:** Per mantenere una visibilità completa sul sistema distribuito, è mandatorio che ogni nuovo microservizio aggiunto all'architettura sia configurato nativamente per l'esportazione delle metriche e delle tracce tramite *OpenTelemetry*. L'assenza di tale configurazione creerebbe dei "punti ciechi" nel sistema di osservabilità, rendendo difficile il debugging distribuito.
- **Ottimizzazione delle prestazioni (Scaling e Orchestrazione):**
Al fine di garantire una migliore esperienza utente e ridurre la latenza in condizioni di traffico elevato, si raccomanda l'implementazione della scalabilità orizzontale per i servizi **t7** e **t8**. Tale evoluzione richiede l'adozione di una piattaforma di orchestrazione con-

tainer (come **Kubernetes**), essenziale per gestire il ciclo di vita delle repliche e garantire un efficace *load balancing* del traffico tra le diverse istanze dei servizi, superando i limiti di una gestione manuale.

- **Possibili vulnerabilità:** È stata identificata una potenziale criticità di sicurezza relativa agli endpoint di gestione */actuator*. Attualmente, tali rotte non richiedono la validazione tramite token JWT, esponendo potenzialmente metriche sensibili e stato del sistema a accessi non autorizzati. Si raccomanda prioritariamente di estendere la configurazione di sicurezza per proteggere tali endpoint.
- **Raffinamento della visualizzazione dati (Grafana):** Sebbene il sistema di monitoraggio sia attivo, sono state riscontrate alcune imprecisioni nella visualizzazione degli errori su Grafana. È necessario un intervento di revisione sulle query delle dashboard per garantire che tutte le tipologie di eccezioni sollevate dai microservizi vengano intercettate e rappresentate correttamente nei grafici, evitando falsi negativi nel monitoraggio della salute del sistema.

Bibliografia

Tomas Cerny, Amr S. Abdelfattah, Abdullah Al Maruf, Andrea Janes, and Davide Taibi. Catalog and detection techniques of microservice anti-patterns and bad smells: A tertiary study. *The Journal of Systems & Software*, 2023. doi: 10.1016/j.jss.2023.111829.

Mark Nottingham and Erik Wilde. Problem Details for HTTP APIs. RFC 9457, RFC Editor, July 2023. URL <https://www.rfc-editor.org/info/rfc9457>. Obsoletes RFC 7807.

Rafik Tighilt, Manel Abdellatif, Imen Trabelsi, Loïc Madern, Naouel Moha, and Yann-Gaël Guéhéneuc. On the maintenance support for microservice-based systems through the specification and the detection of microservice antipatterns. *The Journal of Systems & Software*, 2023. doi: 10.1016/j.jss.2023.111755.